Лабораторная работа №2

Основы вероятностного моделирования угроз

Аристова Арина Олеговна

Содержание

1 Цель работы **4**

2 Задание **5**

3 Теоретическое введение **6**

4 Выполнение лабораторной работы **7**

4.1 Создание и инициализация проекта DrWatson 7
4.2 Ядро моделирования: Файл: src/simulation.jl 8
4.3 Запуск эксперимента с сохранением результатов: однократный запуск эксперимента 10
4.4 Анализ и визуализация результатов одного эксперимента 12
4.5 Исследование сходимости оценки вероятности 14
4.6 Многовариантный эксперимент: параметрическое исследование 16
4.7 Литературный стиль 20

5 Контрольные вопросы **23**

6 Дополнительные задания **27**

7 Выводы **52**

Список литературы **53**

Список иллюстраций

4.1 Инициализация проекта DrWatson 7
4.2 Установка необходимых пакетов 8
4.3 Файл src/simulation.jl 9
4.4 Однократный эксперимент 11
4.5 Выполнение run_experiment.jl 12
4.6 Анализ и визуализация: файл analyze.jl 13
4.7 Выполнение analyze.jl 13
4.8 График для однократного эксперимента 14
4.9 Исследование сходимости оценки вероятности: файл convergence.jl 15
4.10 Результат выполнения convergence.jl 15
4.11 График - результат выполнения convergence.jl 16
4.12 Многовариантный эксперимент: файл parameter_sweep.jl 17
4.13 Выполнение файла parameter_sweep.jl 17
4.14 Созданные файлы в результате выполнения parameter_sweep.jl . 18
4.15 Детальные графики 18
4.16 Детальные графики 18
4.17 Детальные графики 19
4.18 Детальные графики 19
4.19 Детальные графики 19
4.20 График - результат выполнения parameter_sweep.jl 20
4.21 Содержимое файла tangle.jl 20
4.22 Созданные файлы 21
4.23 Jupyter nootebook 22

6.1 Проведение эксперимента с различными параметрами 27
6.2 Код решения задания 2 28
6.3 Результат выполнения кода 29
6.4 Код решения задания 3. 0 атак 30
6.5 Код решения задания 3. Не менее 3 атак за 30 минут 31
6.6 Результат выполнения кода 31
6.7 Код решения задания 4. Бутстрап. Поиск ДИ 32
6.8 Результат выполнения кода задания 4 33
6.9 Получаенный график с отмеченным ДИ 33
6.10 Код задания 5.Вероятность успешности атаки 33
6.11 Получаенный график с вероятностью успешности атаки 34

1 Цель работы

Освоить базовые методы вероятностного моделирования случайных процессов в контексте кибербезопасности.

На примере моделирования потока атак на веб-сервер изучить:

- генерацию случайных величин с заданным распределением;
- статистический анализ смоделированных данных;
- проверку соответствия эмпирического распределения теоретическому;
- оценку вероятностей редких событий;
- определить набор параметров модели (интенсивность атак, длительность наблюдения и т.д.);
- реализовать симуляцию пуассоновского потока атак;
- выполнить статистический анализ результатов;
- визуализировать данные и проверить соответствие теоретическим распределениям.

2 Задание

1. Создать проект DrWatson с именем project.
2. Определить набор параметров модели (словарь params) с возможностью варьирования.
3. Написать функцию simulate_attacks(params), которая возвращает структуру с результатами симуляции (например, массив числа атак по часам, интервалы, моменты времени).
4. Использовать produce_or_load для запуска симуляции с заданными параметрами и сохранения результатов на диск.
5. Провести серию экспериментов с разными значениями параметров (например, разные λ, разная длительность).
6. Загрузить сохранённые результаты и выполнить:
   - построение гистограммы числа атак за час и сравнение с теоретическим распределением Пуассона;
   - построение графика накопленного числа атак и интервалов между атаками;
   - проверку экспоненциальности интервалов (гистограмма, QQ-plot, возможно критерий согласия);
   - оценку вероятности события «более 10 атак за час» теоретически и эмпирически.
7. Проанализировать зависимость точности оценки вероятности от числа симуляций.

3 Теоретическое введение

Во многих задачах кибербезопасности поток атак (или инцидентов) можно приближённо считать простейшим (пуассоновским) потоком. Для такого потока число событий N_t за время t распределено по закону Пуассона:

$$P(N_t = k) = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \quad k = 0, 1, 2, \ldots$$

где λ — интенсивность потока (среднее число событий в единицу времени).

В простейшем потоке интервалы времени между соседними событиями $\tau_1, \tau_2, \ldots$ независимы и имеют экспоненциальное распределение с параметром λ:

$$f(\tau) = \lambda e^{-\lambda \tau}, \quad \tau \geq 0$$

Среднее значение интервала: $\mathbb{E}[\tau] = 1/\lambda$.

4 Выполнение лабораторной работы

Представленные скрипты образуют исследовательский конвейер:

- Моделирование (simulation.jl) — вычислительное ядро.
- Генерация данных (run_experiment.jl) — получение реализаций для фиксированных параметров.
- Анализ и визуализация (analyze.jl) — проверка соответствия модели теоретическим распределениям.
- Исследование сходимости (convergence.jl) — изучение точности оценок.
- Параметрическое исследование (parameter_sweep.jl) — анализ чувствительности к изменению интенсивности.

Для выполнения работы необходимо создать проект с помощью DrWatson. Затем создадим в этом проекте скрипт src/simulation.jl, который содержит функцию simulate_attacks, реализующую вероятностную модель потока атак.

4.1 Создание и инициализация проекта DrWatson

```
julia> initialize_project("project"; authors="Арина Аристова", git=false)
  Activating new project at `C:\Users\arist\Github\2026-1--study--security-m
odeling\labs\lab02\project\project`
```

Рисунок 4.1: Инициализация проекта DrWatson

```
julia> Pkg.add("DataFrames")
   Resolving package versions...
    Updating `C:\Users\arist\Github\2026-1--study--security-modeling\labs\la
b02\project\project\Project.toml`
  [a93c6f00] + DataFrames v1.8.1
    Updating `C:\Users\arist\Github\2026-1--study--security-modeling\labs\la
b02\project\project\Manifest.toml`
  [a8cc5b0e] + Crayons v4.1.1
  [a93c6f00] + DataFrames v1.8.1
  [e2d170a0] + DataValueInterfaces v1.0.0
  [842dd82b] + InlineStrings v1.4.5
  [41ab1584] + InvertedIndices v1.3.1
  [2dfb63ee] + PooledArrays v1.4.3
  [08abe8d2] + PrettyTables v3.2.3
  [91c51154] + SentinelArrays v1.4.9
  [892a3eda] + StringManipulation v0.4.4
  [3783bdb8] + TableTraits v1.0.1
  [bd369af6] + Tables v1.12.1

julia> Pkg.add("Literate")
   Resolving package versions...
    Updating `C:\Users\arist\Github\2026-1--study--security-modeling\labs\la
b02\project\project\Project.toml`
  [98b081ad] + Literate v2.21.0
    Updating `C:\Users\arist\Github\2026-1--study--security-modeling\labs\la
```

Рисунок 4.2: Установка необходимых пакетов

4.2 Ядро моделирования: Файл: src/simulation.jl

Создаём файл src/simulation.jl, функция simulate_attacks реализует вероятностную модель потока атак.

```julia
using Distributions
using Statistics

function simulate_attacks(λ::Float64, T::Float64)
    hourly_counts = rand(Poisson(λ), floor(Int, T)) # Моделирование числа
    intervals = Float64[] # Моделирование интервалов между атаками (точные
    total_time = 0.0
    while total_time < T
        τ = rand(Exponential(1/λ))
        push!(intervals, τ)
        total_time += τ
    end

    # Удаляем последнее событие, если оно вышло за пределы T
    if total_time > T
        pop!(intervals)
    end
        attack_times = cumsum(intervals)
        return (hourly_counts = hourly_counts, intervals = intervals, atta
    end

    # Обёртка для вызова из скрипта с параметром-словарём
    function simulate_attacks(p::Dict)
        return simulate_attacks(p[:λ], p[:T])
end
```

Рисунок 4.3: Файл src/simulation.jl

Эта функция:

- Принимает интенсивность потока λ (среднее число атак в час) и длительность наблюдения T (в часах).
- Генерирует две реализации потока:
  - Почасовое число атак (hourly_counts) — массив длины floor(Int, T), каждый элемент — случайное число из распределения Пуассона с параметром λ.
  - Точные моменты атак — моделирование экспоненциальных интервалов между событиями до тех пор, пока их сумма не превысит T. Возвращает массив интервалов intervals и массив моментов времени attack_times (накопленные суммы).
- Возвращает NamedTuple с тремя полями: hourly_counts, intervals,

attack_times.

Результат:

- Функция не сохраняет данные самостоятельно, а лишь возвращает структуру, которая затем используется в скриптах для дальнейшего анализа или сохранения.

4.3 Запуск эксперимента с сохранением результатов: однократный запуск эксперимента

Файл *scripts/run_experiment.jl*, который выполняет симуляцию с заданными параметрами, вычисляет эмпирическую вероятность события «более 10 атак за час» и сохраняет все результаты на диск. Скрипт предназначен для первичной генерации данных, которые потом будут анализироваться.

```julia
using DrWatson
@quickactivate "project"
using Distributions
using Statistics
using JLD2
include(srcdir("simulation.jl"))
params = Dict(
    :λ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
)
function run_simulation(p)
    @unpack λ, T, num_hours_for_est = p
    res = simulate_attacks(λ, T)
    hourly_sample = rand(Poisson(λ), num_hours_for_est)
    emp_prob = count(hourly_sample .> 10) / num_hours_for_est
    theor_prob = 1 - cdf(Poisson(λ), 10)
    return Dict(
        :hourly_counts => res.hourly_counts,
        :intervals => res.intervals,
        :attack_times => res.attack_times,
        :emp_prob => emp_prob,
        :theor_prob => theor_prob
    )
end

# Генерируем имя файла на основе параметров (функция safename из DrWatson)
filename = datadir("attack_sim", savename(params, "jld2"))
mkpath(datadir("attack_sim"))
# Проверяем, существует ли уже файл
if isfile(filename)
    println("Загрузка существующих данных из $filename")
    data = load(filename)["data"]
else
    println("Запуск симуляции...")
```

Рисунок 4.4: Однократный эксперимент

Файл вызывает simulate_attacks(λ, T) для получения почасовых counts, интервалов и моментов времени, дополнительно генерирует большую выборку (num_hours_for_est) из распределения Пуассона для устойчивой оценки эмпирической вероятности □(□1 > 10), вычисляет теоретическую вероятность 1 − □Poisson(λ)(10), упаковывает все результаты в словарь и сохраняет в JLD2-файл (команда **save**).

Результат:

— В папке data/attack_sim/ создаётся файл с именем, отражающим параметры (например, attack_sim_λ=5.0_T=24.0_num_hours_for_est=10000.jld2).

— Внутри — словарь с ключами: :hourly_counts, :intervals, :attack_times,

:emp_prob, :theor_prob.

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project> julia C:\Users\ar
ist\Github\2026-1--study--security-modeling\labs\lab02\project\scripts\run_experiment.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\proj
ect`
ist\Github\2026-1--study--security-modeling\labs\lab02\project\scripts\run_experiment.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\proj
ect`
Запуск симуляции...
Сохраняем в файл...
Результаты сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\projec
t\data\attack_sim\T=24.0_num_hours_for_est=10000_λ=5.0.jld2
Эмпирическая вероятность P(>10) = 0.0133
Теоретическая вероятность = 0.013695268598382881
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project>
```

Рисунок 4.5: Выполнение run_experiment.jl

4.4 Анализ и визуализация результатов одного эксперимента

Файл *scripts/analyze.jl* загружает сохранённые данные и строит четыре диагностических графика, позволяющих визуально оценить соответствие смоделированного потока теоретическим предположениям (пуассоновости и экспоненциальности).

```julia
using DrWatson
@quickactivate "project"
using Plots, Distributions, StatsPlots, JLD2
# Определяем параметры эксперимента (должны совпадать с использованными в
params = Dict(
    :λ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
)

# Формируем путь к файлу с данными (как в run_experiment.jl)
filename = datadir("attack_sim", savename(params, "jld2"))

# Проверяем существование файла
if !isfile(filename)
    error("Файл $filename не найден. Сначала запустите scripts/run_experim
end

# Загружаем данные
@load filename data
# data — это словарь (Dict), сохранённый в run_experiment.jl
# Извлекаем результаты
hourly_counts = data[:hourly_counts]
intervals = data[:intervals]
attack_times = data[:attack_times]
λ = params[:λ]
T = params[:T]
emp_prob = data[:emp_prob]
theor_prob = data[:theor_prob]
```

Рисунок 4.6: Анализ и визуализация: файл analyze.jl

Результат:

— PNG-файл с четырьмя графиками, наглядно демонстрирующими свойства сгенерированного потока.

— В консоль выводятся значения эмпирической и теоретической вероятностей

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project> julia C:\Users\ar
ist\Github\2026-1--study--security-modeling\labs\lab02\project\scripts\analyze.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\proj
ect`
Эмпирическая вероятность P(>10) = 0.0133
Теоретическая вероятность = 0.013695268598382881
Графики сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\p
lots\attack_sim_plots.png
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project>
```

Рисунок 4.7: Выполнение analyze.jl

Получаем сводный график для однократного эксперимента.

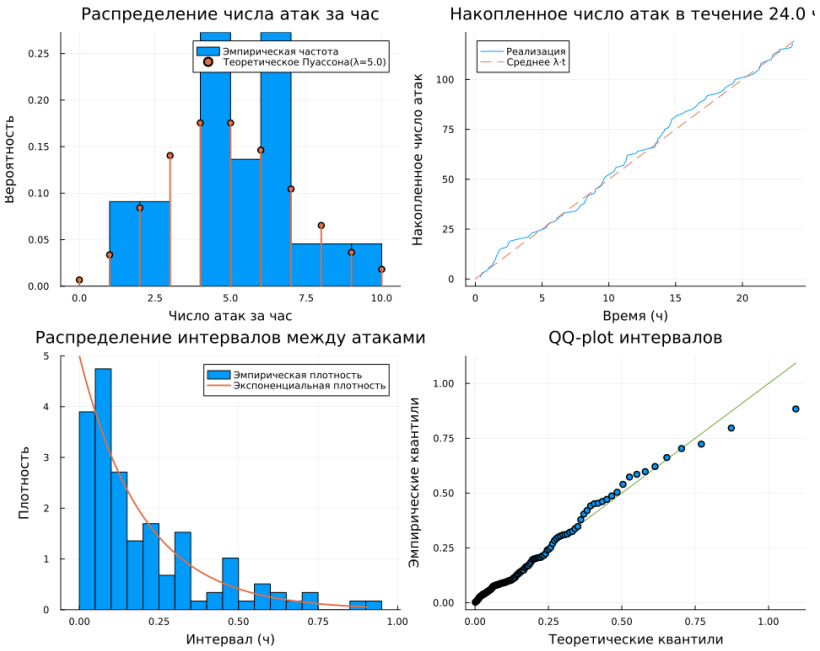
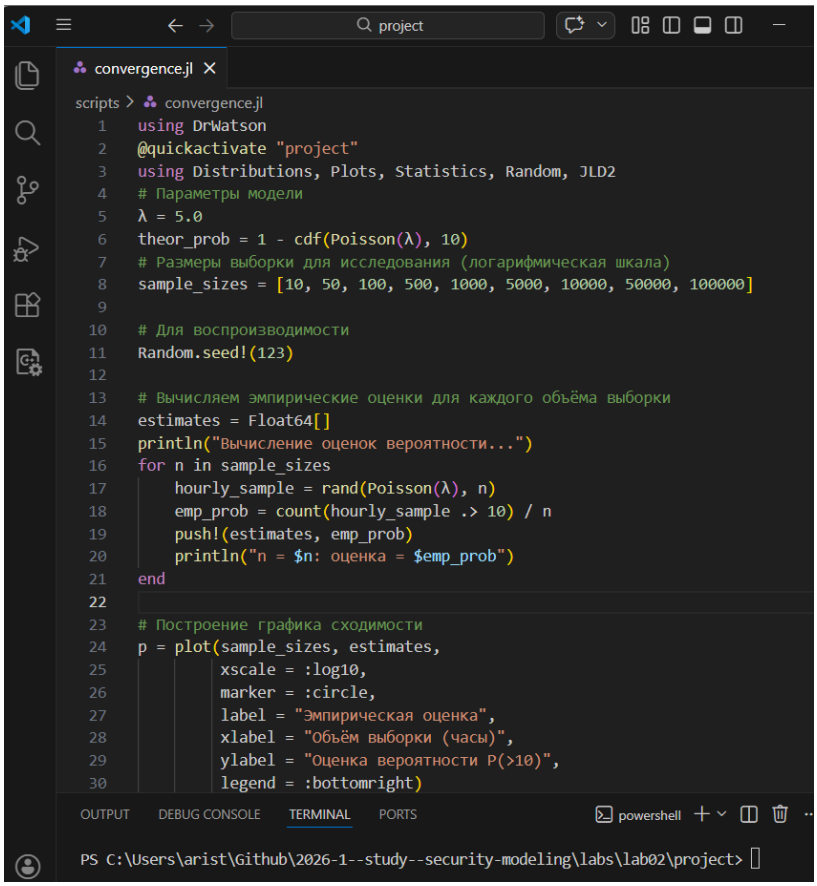


Рисунок 4.8: График для однократного эксперимента

4.5 Исследование сходимости оценки вероятности

Файл *scripts/convergence.jl* позволяет изучить, как быстро эмпирическая оценка вероятности редкого события приближается к теоретическому значению при увеличении объёма выборки.Это важный практический аспект моделирования: для надёжной оценки редких событий требуется большой объём данных.



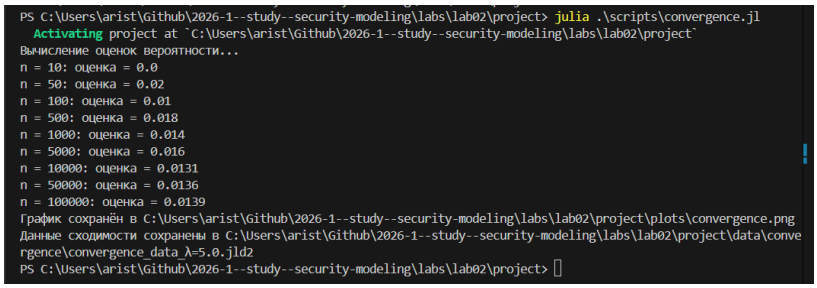
```julia
using DrWatson
@quickactivate "project"
using Distributions, Plots, Statistics, Random, JLD2
# Параметры модели
λ = 5.0
theor_prob = 1 - cdf(Poisson(λ), 10)
# Размеры выборки для исследования (логарифмическая шкала)
sample_sizes = [10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]

# Для воспроизводимости
Random.seed!(123)

# Вычисляем эмпирические оценки для каждого объёма выборки
estimates = Float64[]
println("Вычисление оценок вероятности...")
for n in sample_sizes
    hourly_sample = rand(Poisson(λ), n)
    emp_prob = count(hourly_sample .> 10) / n
    push!(estimates, emp_prob)
    println("n = $n: оценка = $emp_prob")
end

# Построение графика сходимости
p = plot(sample_sizes, estimates,
         xscale = :log10,
         marker = :circle,
         label = "Эмпирическая оценка",
         xlabel = "Объём выборки (часы)",
         ylabel = "Оценка вероятности P(>10)",
         legend = :bottomright)
```

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project>
```

Рисунок 4.9: Исследование сходимости оценки вероятности: файл convergence.jl

Результат:

— График, иллюстрирующий уменьшение случайной ошибки оценки с ростом выборки, и файл с данными для повторного построения.



```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project> julia .\scripts\convergence.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
Вычисление оценок вероятности...
n = 10: оценка = 0.0
n = 50: оценка = 0.02
n = 100: оценка = 0.01
n = 500: оценка = 0.018
n = 1000: оценка = 0.014
n = 5000: оценка = 0.016
n = 10000: оценка = 0.0131
n = 50000: оценка = 0.0136
n = 100000: оценка = 0.0139
График сохранён в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\convergence.png
Данные сходимости сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\conve
rgence\convergence_data_λ=5.0.jld2
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project>
```

Рисунок 4.10: Результат выполнения convergence.jl

График, иллюстрирующий уменьшение случайной ошибки оценки с ростом

выборки, и файл с данными для повторного построения.

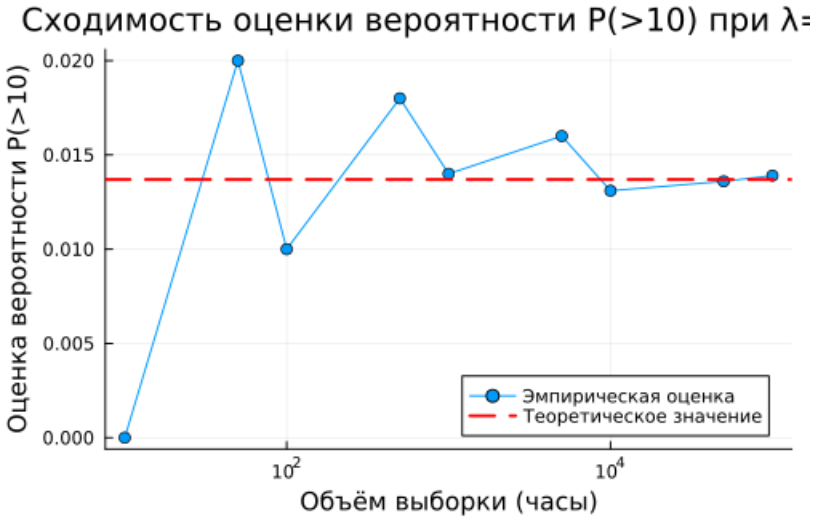


Рисунок 4.11: График - результат выполнения convergence.jl

4.6 Многовариантный эксперимент: параметрическое исследование

Файл *scripts/parameter_sweep.jl* позволяет систематически изучить, как изменение интенсивности атак λ влияет на характеристики потока и, в частности, на вероятность □(> 10). Скрипт автоматизирует запуск множества экспериментов, сохраняет все результаты и строит как обобщающий график, так и детальные графики для каждого значения λ.

```julia
using DrWatson
@quickactivate "project"
mkpath(datadir("parameter_sweep"))

using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV, DataFrames
# Включаем функцию симуляции
include(srcdir("simulation.jl"))
# Базовые параметры (общие для всех запусков)
base_params = Dict(
    :T => 24.0,
    :num_hours_for_est => 10000
)

# Диапазон значений λ для исследования
λ_values = [2.0, 5.0, 8.0, 12.0, 15.0]

# Для воспроизводимости
Random.seed!(42)

# Создаём папку для графиков параметрического исследования, если её нет
parametric_plots_dir = plotsdir("parameter_sweep")
mkpath(parametric_plots_dir)
# Структура для хранения сводных результатов
summary = Dict{Float64, Dict}()

println("Запуск параметрического исследования...")
for λ in λ_values
    # Формируем полный словарь параметров
    params = merge(base_params, Dict(:λ => λ))

    # Генерируем имя файла на основе параметров
    filename = datadir("attack_sim", savename(params, "jld2"))

    # Проверяем, существует ли уже файл с результатами
    if isfile(filename)
```

Рисунок 4.12: Многовариантный эксперимент: файл parameter_sweep.jl

Выполняем этот файл. Результат:

— Набор JLD2-файлов для каждого λ в data/attack_sim/.

— Детальные графики для каждого λ в plots/parameter_sweep/.

— Сводная таблица (CSV) и файл с данными в data/parameter_sweep/.

— Обобщающий график в корне plots/, показывающий зависимость вероятности от интенсивности.

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project> julia .\scripts\parameter_sweep.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
Запуск параметрического исследования...
Загрузка существующих данных для λ = 2.0 из C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est=10000_λ=2.0.jld2
Детальные графики для λ=2.0 сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_λ=2.0.png
λ = 2.0: теоретическая вероятность = 8.308224368480666e-6, эмпирическая = 0.0
Загрузка существующих данных для λ = 5.0 из C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est=10000_λ=5.0.jld2
Детальные графики для λ=5.0 сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_λ=5.0.png
λ = 5.0: теоретическая вероятность = 0.013695268598382881, эмпирическая = 0.0133
Загрузка существующих данных для λ = 8.0 из C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est=10000_λ=8.0.jld2
Детальные графики для λ=8.0 сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_λ=8.0.png
λ = 8.0: теоретическая вероятность = 0.18411420744145301, эмпирическая = 0.1822
Загрузка существующих данных для λ = 12.0 из C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est=10000_λ=12.0.jld2
Детальные графики для λ=12.0 сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_λ=12.0.png
λ = 12.0: теоретическая вероятность = 0.65277058245858284, эмпирическая = 0.6505
Загрузка существующих данных для λ = 15.0 из C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est=10000_λ=15.0.jld2
Детальные графики для λ=15.0 сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_λ=15.0.png
λ = 15.0: теоретическая вероятность = 0.8815355884709849, эмпирическая = 0.8809
Сводные данные сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\parameter_sweep\summary_λ_values.jld2
Общий график зависимости сохранён в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\parameter_sweep.png
Таблица сохранена в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\parameter_sweep\summary.csv
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project>
```

Рисунок 4.13: Выполнение файла parameter_sweep.jl

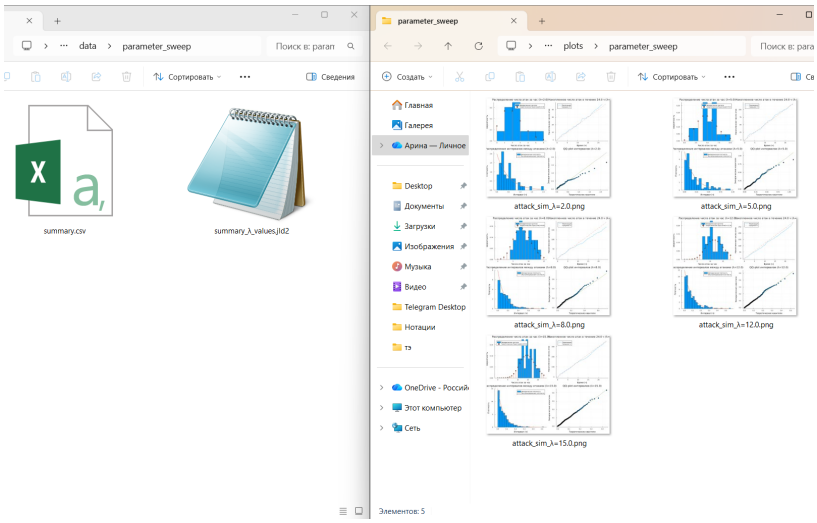


Рисунок 4.14: Созданные файлы в результате выполнения parameter_sweep.jl

Детальные графики для каждого λ в plots/parameter_sweep/.

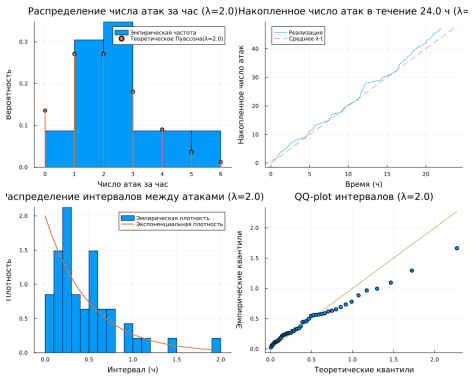


Рисунок 4.15: Детальные графики

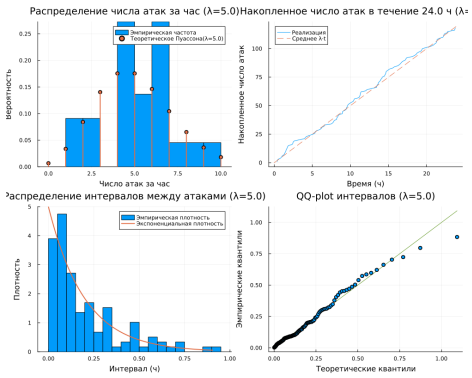


Рисунок 4.16: Детальные графики

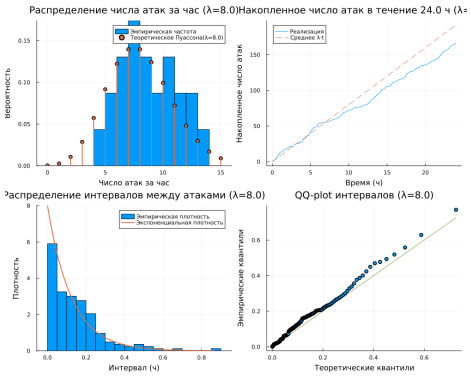


Рисунок 4.17: Детальные графики

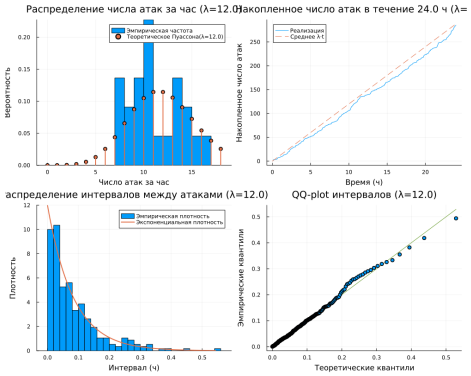


Рисунок 4.18: Детальные графики

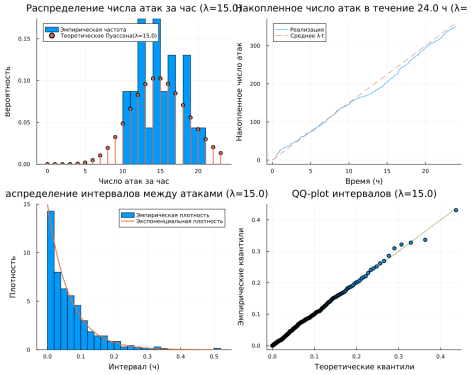


Рисунок 4.19: Детальные графики

А также получаем график, показывающий зависимость вероятности от интенсивности.

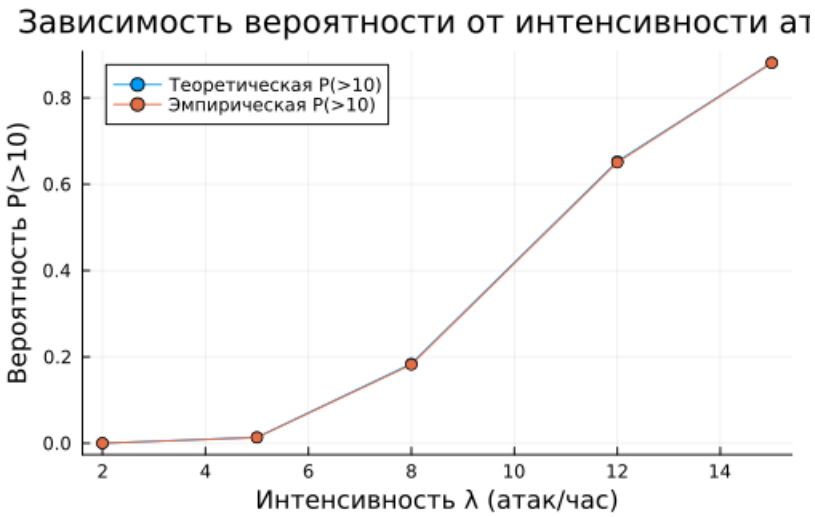


Рисунок 4.20: График - результат выполнения parameter_sweep.jl

4.7 Литературный стиль

Преобразовываем в производные форматы с помощью файла *tangle.jl*

```
tangle.jl
scripts > tangle.jl
using DrWatson
@quickactivate "project"
using Literate

mkpath(joinpath(@__DIR__, "..", "notebooks"))
mkpath(joinpath(@__DIR__, "..", "markdown"))

script1 = joinpath(@__DIR__, "run_experiment.jl")
script2 = joinpath(@__DIR__, "convergence.jl")
script3 = joinpath(@__DIR__, "parameter_sweep.jl")

for (script, name) in [(script1, "run_experiment"),
                       (script2, "convergence"),
                       (script3, "parameter_sweep")]
    Literate.script(  script, @__DIR__;
        name = name * "_clean")
    Literate.notebook(script, joinpath(@__DIR__, "..", "notebooks");
        name = name, execute = false)
    Literate.markdown(script, joinpath(@__DIR__, "..", "markdown");
        name = name, flavor = Literate.QuartoFlavor())
end

println("Готово.")
```

Рисунок 4.21: Содержимое файла tangle.jl

В результате выполнения файла *tangle.jl* создались следующие файлы:

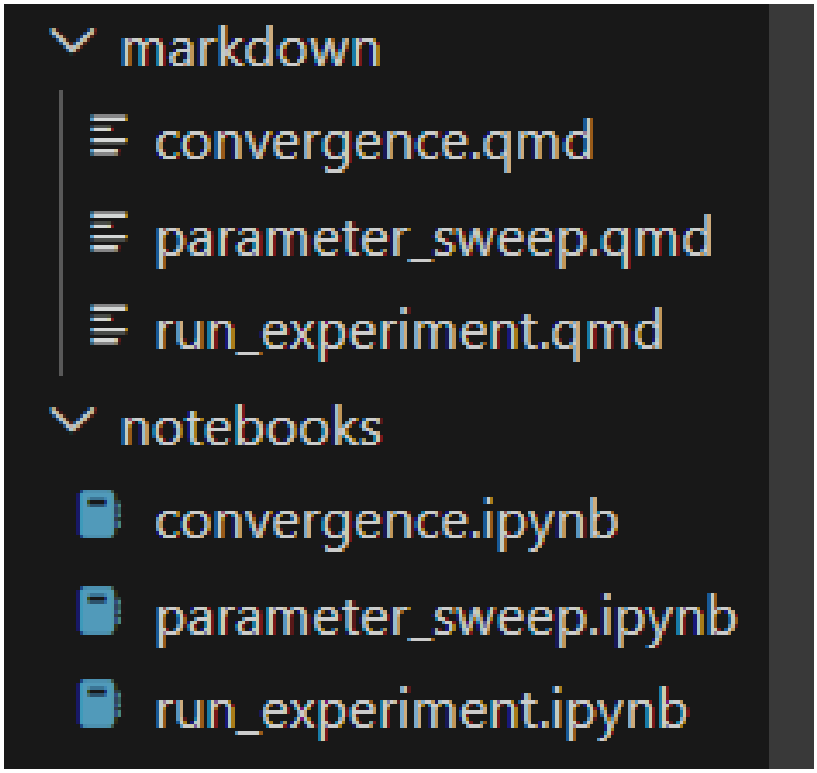


Рисунок 4.22: Созданные файлы

Посмотрим и выполним jupyter nootebook -и.

tangle.jl parameter_sweep.ipynb

notebooks > parameter_sweep.ipynb > using DrWatson

Generate Code Markdown Run All Restart Clear All Outputs Outline

```
using DrWatson
@quickactivate "project"
mkpath(datadir("parameter_sweep"))

using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV,
```

[1]

Включаем функцию симуляции

```
include(srcdir("simulation.jl"))
```

[2]

```
simulate_attacks (generic function with 2 methods)
```

Базовые параметры (общие для всех запусков)

```
base_params = Dict(
    :T => 24.0,
    :num_hours_for_est => 10000
)
```

[3]

```
Dict{Symbol, Real} with 2 entries:
  :T                 => 24.0
  :num_hours_for_est => 10000
```

Диапазон значений λ для исследования

Рисунок 4.23: Jupyter nootebook

5 Контрольные вопросы

1. Какими свойствами обладает простейший поток событий? Почему он часто используется для моделирования атак?

Простейший (пуассоновский) поток обладает тремя свойствами:

1. **Стационарность** — вероятность появления k событий за интервал длины t зависит только от t и не зависит от момента начала отсчёта.

2. **Отсутствие последействия** — число событий в непересекающихся интервалах независимо.

3. **Ординарность** — вероятность появления двух и более событий за бесконечно малый интервал пренебрежимо мала.

Используется для моделирования атак потому что это математически простой и хорошо изученный процесс с готовыми аналитическими формулами. Согласно теореме Пальма–Хинчина, суперпозиция большого числа независимых стационарных ординарных потоков сходится к пуассоновскому — что соответствует реальной ситуации, когда атаки приходят от множества независимых источников.

2. Как сгенерировать реализацию пуассоновского потока на интервале времени?

Используется метод на основе экспоненциального распределения интервалов. Алгоритм:

1. Задать интенсивность λ и длительность T.

2. Установить t = 0.

3. Генерировать τ ~ Exp(λ), прибавлять к t.

4. Если t ≤ T — записать момент события, иначе — остановиться.

3. Как проверить, что интервалы между событиями распределены экспоненциально?

Три способа:

- **Гистограмма с теоретической плотностью** — строим гистограмму интервалов и накладываем кривую Exp(λ). Если совпадают — распределение экспоненциальное.

- **QQ-plot** — строим квантиль-квантильный график интервалов против теоретического экспоненциального распределения. В нашем коде это `qqplot(Exponential(1/$\lambda$), intervals, qqline = :identity)`. Если точки лежат на прямой y = x — распределение экспоненциальное.

- **Критерий Колмогорова–Смирнова** — если p-value > 0.05, нет оснований отвергать гипотезу об экспоненциальности.

4. В чём преимущества использования DrWatson для организации вычислительного эксперимента?

- **Стандартная структура проекта** — папки `src/`, `scripts/`, `data/`, `plots/` создаются автоматически.

- **Воспроизводимость** — проект инициализируется как Julia-окружение с `Project.toml` и `Manifest.toml`, фиксируя версии всех пакетов.

- **`@quickactivate`** — макрос гарантирует что скрипт выполняется в контексте именно этого проекта.

- **`savename`** — автоматически формирует имя файла из словаря параметров, исключая путаницу между результатами разных запусков.
- **Кэширование** — `produce_or_load` не пересчитывает результаты если файл уже существует.

5. Что такое `produce_or_load` и как он работает?

`produce_or_load` — функция DrWatson для кэширования результатов вычислений. Логика работы:

1. Формирует путь к файлу на основе параметров через `savename`.
2. Если файл существует — загружает данные с диска, вычисление не запускается.
3. Если файла нет — выполняет вычисление и сохраняет результат.

В нашей лабораторной работе реализована та же логика вручную через `isfile` + `@save`/`@load`, что эквивалентно `produce_or_load`.

6. Какая структура проекта создаётся DrWatson? Для чего нужны папки?

```
project/
├── data/         # результаты симуляций, сохранённые JLD2/CSV файлы
├── scripts/      # запускаемые скрипты
├── src/          # исходный код — функции и модули
├── plots/        # сохранённые графики
├── notebooks/    # Jupyter notebooks
├── markdown/     # Quarto-документация
├── Project.toml  # список зависимостей проекта
└── Manifest.toml # зафиксированные версии всех пакетов
```

`src/` — переиспользуемый код, подключается через `include(srcdir(...))`. `scripts/` — точки входа, запускаются напрямую. `data/` — только результаты, не редактируется вручную. `plots/` — только графики, генерируются скриптами.

7. Как задать набор параметров для множественных запусков в DrWatson?

Вынести параметры в отдельный файл `scripts/params.jl`:

```
default_params = Dict(
    :$\lambda$ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
)
```

В других скриптах подключать и переопределять нужные поля через `merge`. Для множественных запусков — создавать список словарей и итерировать по нему, как реализовано в `parameter_sweep.jl` через цикл по `$\lambda$_values`.

6 Дополнительные задания

1. Изменить интенсивность λ (например, 2, 8, 12 атак/час) и сравнить результаты.

- При λ=2 распределение числа атак сдвинуто влево — пик около 1-2 атак за час, эмпирическая гистограмма хорошо совпадает с теоретическим Пуассоном(2). Интервалы между атаками большие — около 0.3-2 ч. Вероятность P(>10) близка к нулю (~$8.3 \cdot 10^{-6}$).
- При λ=8 распределение смещается вправо — пик около 7-8 атак за час, интервалы короткие — большинство менее 0.2 ч. Вероятность P(>10) резко возрастает (~0.184).

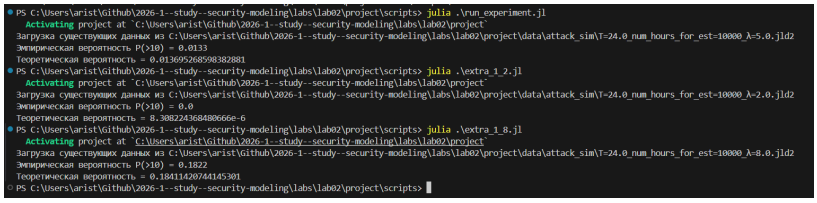


Рисунок 6.1: Проведение эксперимента с различными параметрами

2. Моделировать нестационарный пуассоновский поток с интенсивностью, зависящей от времени суток: $\lambda(t) = 2 + 5sin(\pi t/12)$. Модифицировать функцию симуляции (использовать метод прореживания или неоднородный пуассоновский процесс).

lab02

security-modeling-lab02-report.qmd | extra_2.jl

project > scripts > extra_2.jl

```julia
using DrWatson
@quickactivate "project"
using Distributions, Plots, Random

Random.seed!(42)

# Нестационарный поток: интенсивность меняется в течение суток.
# Минимум : t=0 или t=24 и максимум днём t=12.
λ_func(t) = 2.0 + 5.0 * sin(π * t / 12.0)
λ_max = 7.0  # максимальное значение λ(t) на [0, 24]
T = 24.0

# Метод прореживания:
# 1. Генерируем однородный поток с λ_max (верхняя граница).
# 2. Каждое событие в момент t принимаем с вероятностью λ(t)/λ_max.
# 3. Отвергнутые события просто удаляются.
# Результат — нестационарный поток с интенсивностью λ(t).
function simulate_nonstationary(λ_func, λ_max, T)
    attack_times = Float64[]
    t = 0.0
    while t < T
        τ = rand(Exponential(1 / λ_max))
        t += τ
        t > T && break
        if rand() < λ_func(t) / λ_max
            push!(attack_times, t)
        end
    end
    return attack_times
end

attack_times = simulate_nonstationary(λ_func, λ_max, T)

println("Кол-во атак за сутки: ", length(attack_times))

t_grid = 0:0.1:T
p1 = plot(t_grid, λ_func.(t_grid),
    label  = "λ(t) = 2 + 5·sin(πt/12)",
    xlabel = "Время (ч)", ylabel = "Интенсивность",
    lw = 2, title = "Нестационарный поток: интенсивность")

p2 = plot(attack_times, 1:length(attack_times),
    label  = "Реализация",
    xlabel = "Время (ч)", ylabel = "Накопленное число атак",
    title  = "Нестационарный поток: накопленное число атак")
```

Рисунок 6.2: Код решения задания 2

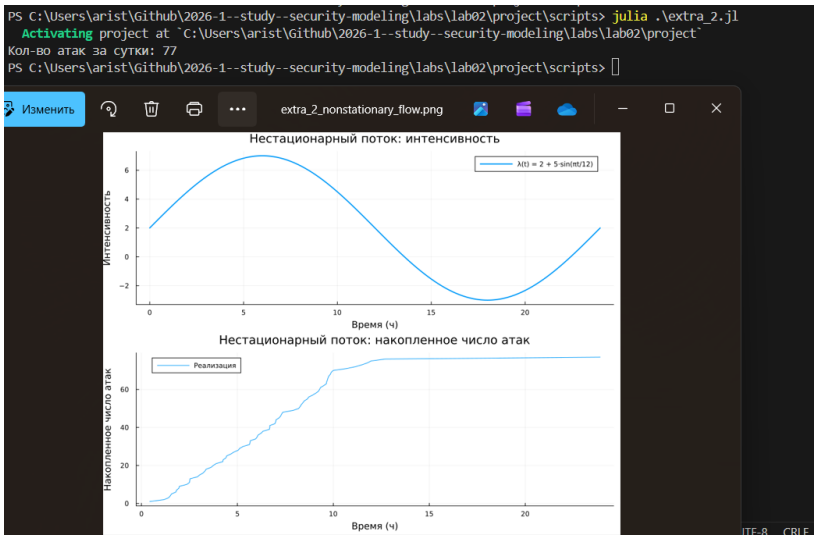


Рисунок 6.3: Результат выполнения кода

3. Исследовать вероятность события «ни одной атаки за смену (8 часов)» или «не менее 3 атак за 30 минут»

lab02

security-modeling-lab02-report.qmd | extra_3_0attacks.jl | extra_2.jl

project > scripts > extra_3_0attacks.jl

```julia
using DrWatson
@quickactivate "project"
using Distributions
using Statistics
using JLD2
include(srcdir("simulation.jl"))

params = Dict(
    :λ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000  # количество 8-часовых смен для оценки
)

function run_simulation(p)
    @unpack λ, T, num_hours_for_est = p

    # Основная симуляция
    res = simulate_attacks(λ, T)

    # Вероятность: ни одной атаки за смену (8 часов)
    # Генерируем много 8-часовых интервалов
    shift_counts = [sum(rand(Poisson(λ), 8)) for _ in 1:num_hours_for_est]
    emp_prob = count(shift_counts .== 0) / num_hours_for_est
    theor_prob = pdf(Poisson(λ * 8), 0)  # P(0 атак за 8 часов)

    return Dict(
        :hourly_counts => res.hourly_counts,
        :intervals => res.intervals,
        :attack_times => res.attack_times,
        :emp_prob => emp_prob,
        :theor_prob => theor_prob
    )
end

# Генерируем имя файла
filename = datadir("attack_sim_0attacks", savename(params, "jld2"))
mkpath(datadir("attack_sim_0attacks"))

# Проверяем существование файла
if isfile(filename)
    println("Загрузка существующих данных из $filename")
    data = load(filename)["data"]
else
    println("Запуск симуляции...")
    data = run_simulation(params)
```

Рисунок 6.4: Код решения задания 3. 0 атак

```julia

Random.seed!(42)

# Нестационарный поток: интенсивность меняется в течение суток.
# Минимум : t=0 или t=24 и максимум днём t=12.
λ_func(t) = 2.0 + 5.0 * sin(π * t / 12.0)
λ_max = 7.0  # максимальное значение λ(t) на [0, 24]
T = 24.0

# Метод прореживания:
# 1. Генерируем однородный поток с λ_max (верхняя граница).
# 2. Каждое событие в момент t принимаем с вероятностью λ(t)/λ_max.
# 3. Отвергнутые события просто удаляются.
# Результат — нестационарный поток с интенсивностью λ(t).
function simulate_nonstationary(λ_func, λ_max, T)
    attack_times = Float64[]
    t = 0.0
    while t < T
        τ = rand(Exponential(1 / λ_max))
        t += τ
        t > T && break
        if rand() < λ_func(t) / λ_max
            push!(attack_times, t)
        end
    end
    return attack_times
end

attack_times = simulate_nonstationary(λ_func, λ_max, T)

println("Кол-во атак за сутки: ", length(attack_times))

t_grid = 0:0.1:T
p1 = plot(t_grid, λ_func.(t_grid),
    label  = "λ(t) = 2 + 5·sin(πt/12)",
    xlabel = "Время (ч)", ylabel = "Интенсивность",
    lw = 2, title = "Нестационарный поток: интенсивность")

p2 = plot(attack_times, 1:length(attack_times),
    label  = "Реализация",
    xlabel = "Время (ч)", ylabel = "Накопленное число атак",
    title  = "Нестационарный поток: накопленное число атак")
# Вертикальные линии показывают моменты атак
vline!(p2, attack_times, label = "Моменты атак", alpha = 0.4, color = :red)

```

Рисунок 6.5: Код решения задания 3. Не менее 3 атак за 30 минут

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\scripts> julia .\extra_3_0attacks.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
Загрузка существующих данных из C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim_0attacks\T=24.0_num_hours_for_est=10000_λ=5.0.jld2
Вероятность: нм одной атаки за 8-часовую смену
Эмпирическая: 0.0
Теоретическая: 4.248354255291589e-18
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\scripts> julia .\extra_3_330.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
Запуск симуляции...
Сохраняем в файл...
Результаты сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\attack_sim_3_for_30min\T=24.0_num_hours_for_est=10000_λ=5.0.jld2
Вероятность: нм одной атаки за 8-часовую смену
Эмпирическая: 0.4544
Теоретическая: 0.4561868841166705
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\scripts>
```

Рисунок 6.6: Результат выполнения кода

4. Оценить доверительный интервал для вероятности редкого события

методом бутстрепа.

С помощью бутстрапа и прореживания потока находим 95-проуентный ДИ и визуализируем.

```
using DrWatson
@quickactivate "project"
using Distributions, Plots, Statistics, Random

Random.seed!(42)

λ = 5.0
threshold = 10
N = 1000
B = 10_000

sample = rand(Poisson(λ), N)
emp_prob = count(sample .> threshold) / N
theor = 1 - cdf(Poisson(λ), threshold)

boot_estimates = Float64[]
for _ in 1:B
    boot_sample = sample[rand(1:N, N)]
    push!(boot_estimates, count(boot_sample .> threshold) / N)
end

ci_low  = quantile(boot_estimates, 0.025)
ci_high = quantile(boot_estimates, 0.975)

println("Эмпирическое значение: ", emp_prob)
println("Теоретическое значение: ", theor)
println("95% ДИ (бутстреп): [$ci_low, $ci_high]")

p = histogram(boot_estimates,
    bins      = 50,
    normalize = :pdf,
    label = "Бутстреп-оценки",
    xlabel = "Оценка P(>$threshold)",
    ylabel = "Плотность",
    title = "Бутстреп-распределение оценки P(>$threshold)")
vline!(p, [theor],
    label = "Теоретическое значение",
    lw = 2, color = :red, ls = :dash)
vline!(p, [ci_low, ci_high],
    label = "Границы 95% ДИ",
    lw = 2,
    color = :green,
    ls = :dot)

display(p)
```

Рисунок 6.7: Код решения задания 4. Бутстрап. Поиск ДИ

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02> julia .\project\scripts\extra_4_bootstra
p.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
Эмпирическое значение: 0.017
Теоретическое значение: 0.013695268598382881
95% ДИ (бутстреп): [0.01, 0.026]
График сохранён в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\extra_4_boot
strap.png
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02>
```

Рисунок 6.8: Результат выполнения кода задания 4

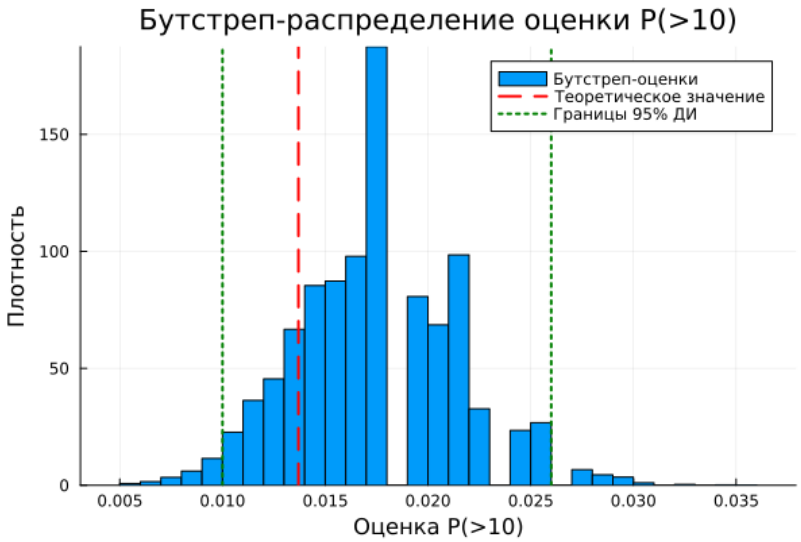


Рисунок 6.9: Получаенный график с отмеченным ДИ

5. Добавить в модель возможность успешности атаки: каждая атака имеет вероятность успеха □, тогда успешные атаки образуют прореженный п

Добавляем вероятность успешности атаки и рассчитываем соответствующие вероятности.

```
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02> julia .\project\scripts\extra_5_success.jl
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
λ исходный:  5.0
p успеха:    0.5
λ успешных:  2.5
Всего атак:    131
Успешных атак: 62
P(>5 успешных/час):
  Теоретическая вероятность =0.042021038195306115
  Эмпирическая вероятность =0.0405
График сохранён в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\extra_5_success.png
PS C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02>
```

Рисунок 6.10: Код задания 5.Вероятность успешности атаки

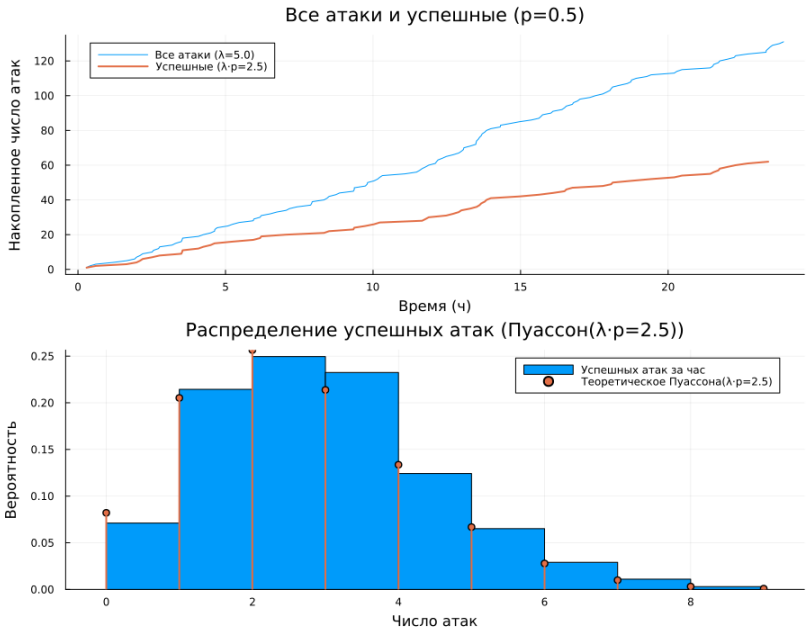


Рисунок 6.11: Получаенный график с вероятностью успешности атаки

```
cd(joinpath(@__DIR__, "../project"))

# Активируем окружение проекта
using Pkg
Pkg.activate(".")
Pkg.instantiate()

# Активируем DrWatson в текущей папке
using DrWatson
@quickactivate "."
using Distributions
using Statistics
using JLD2
include(srcdir("simulation.jl"))
params = Dict(
```

```
    :□ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
)
function run_simulation(p)
    @unpack □, T, num_hours_for_est = p
    res = simulate_attacks(□, T)
    hourly_sample = rand(Poisson(□), num_hours_for_est)
    emp_prob = count(hourly_sample .> 10) / num_hours_for_est
    theor_prob = 1 - cdf(Poisson(□), 10)
    return Dict(
        :hourly_counts => res.hourly_counts,
        :intervals => res.intervals,
        :attack_times => res.attack_times,
        :emp_prob => emp_prob,
        :theor_prob => theor_prob
    )
end
```

```
Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
```

```
run_simulation (generic function with 1 method)
```

Генерируем имя файла на основе параметров (функция safename из DrWatson)

```
filename = datadir("attack_sim", savename(params, "jld2"))
mkpath(datadir("attack_sim"))
```

```
"C:\\Users\\arist\\Github\\2026-1--study--security-modeling\\labs\\lab02\\project\\da
```

Проверяем, существует ли уже файл

```
if isfile(filename)
    println("Загрузка существующих данных из $filename")
    data = load(filename)["data"]
else
    println("Запуск симуляции...")
    data = run_simulation(params)
    println("Сохраняем в файл...")
    @save filename data
    println("Результаты сохранены в $filename")
end
    println("Эмпирическая вероятность P(>10) = ", data[:emp_prob])
    println("Теоретическая вероятность = ", data[:theor_prob])
```

```
Загрузка существующих данных из C:\Users\arist\Github\2026-1--study--
security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_est=10000_[
Эмпирическая вероятность P(>10) = 0.0133
Теоретическая вероятность = 0.013695268598382881
```

```
cd(joinpath(@__DIR__, "../project"))

# Активируем окружение проекта
using Pkg
Pkg.activate(".")
Pkg.instantiate()

# Активируем DrWatson в текущей папке
using DrWatson
```

```julia
@quickactivate "."
using Plots, Distributions, StatsPlots, JLD2
gr(fmt=:png)
ENV["GKSwstype"] = "100"
```

```
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
```

```
"100"
```

Определяем параметры эксперимента (должны совпадать с использованными в run_experiment.jl)

```julia
params = Dict(
    :□ => 5.0,
    :T => 24.0,
    :num_hours_for_est => 10000
)
```

```
Dict{Symbol, Real} with 3 entries:
  :T                 => 24.0
  :□                 => 5.0
  :num_hours_for_est => 10000
```

Формируем путь к файлу с данными (как в run_experiment.jl)

```julia
filename = datadir("attack_sim", savename(params, "jld2"))
```

```
"C:\\Users\\arist\\Github\\2026-1--study--security-modeling\\labs\\lab02\\project\\da
```

Проверяем существование файла

```
if !isfile(filename)
    error("Файл $filename не найден. Сначала запустите scripts/run_experiment.jl для
end
```

Загружаем данные

```
@load filename data
```

```
1-element Vector{Symbol}:
 :data
```

data — это словарь (Dict), сохранённый в run_experiment.jl Извлекаем результаты

```
hourly_counts = data[:hourly_counts]
intervals = data[:intervals]
attack_times = data[:attack_times]
□ = params[:□]
T = params[:T]
emp_prob = data[:emp_prob]
theor_prob = data[:theor_prob]
```

```
0.013695268598382881
```

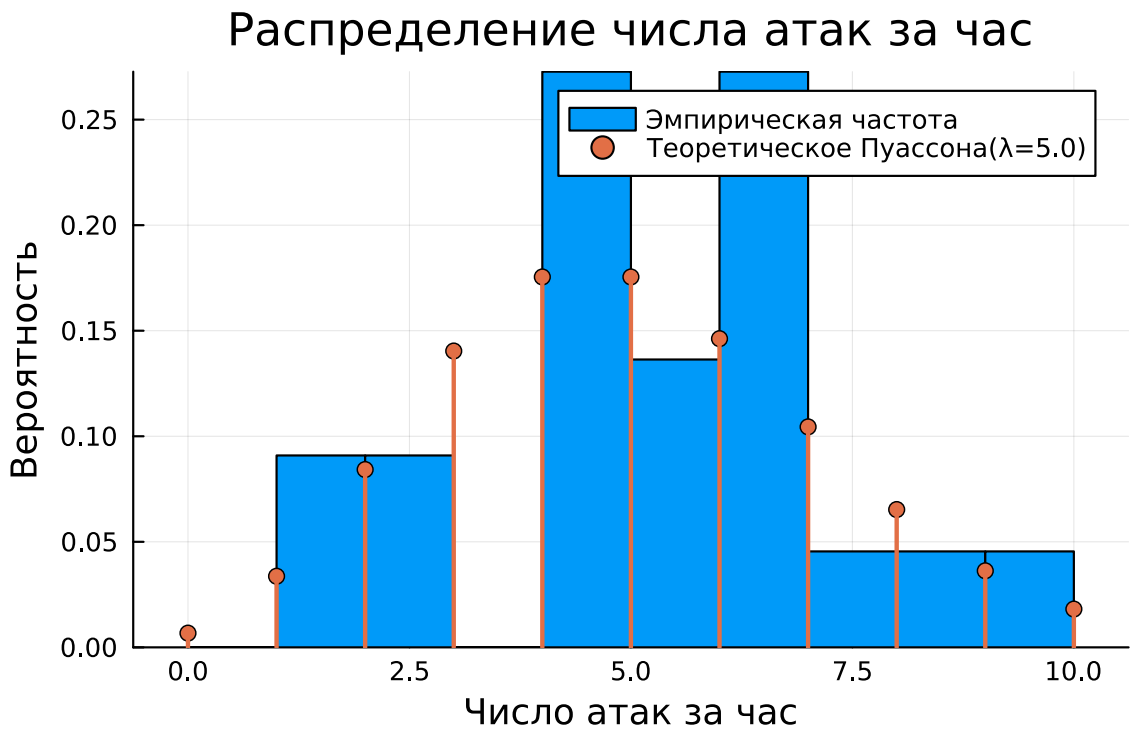
Выводим вероятности для справки

```
println("Эмпирическая вероятность P(>10) = ", emp_prob)
println("Теоретическая вероятность = ", theor_prob)
```

```
Эмпирическая вероятность P(>10) = 0.0133
Теоретическая вероятность = 0.013695268598382881
```

1. Гистограмма числа атак за час

```
p1 = histogram(hourly_counts, bins = 0:maximum(hourly_counts), normalize = :probabili
x_vals = 0:maximum(hourly_counts)
theor_probs = pdf.(Poisson(□), x_vals)
plot!(p1, x_vals, theor_probs, line = :stem, marker = :circle, label = "Теоретическое
title!(p1, "Распределение числа атак за час")
```



2. Накопленное число атак

```
p2 = plot(attack_times, 1:length(attack_times), label = "Реализация", xlabel = "Время

plot!(p2, 0:0.1:T, □*(0:0.1:T), label = "Среднее □·t", ls = :dash)
title!(p2, "Накопленное число атак в течение $(T) ч")
nothing
```

3. Гистограмма интервалов

```
p3 = histogram(intervals, bins = 30, normalize = :pdf, label =  "Эмпирическая плотнос
xlabel = "Интервал (ч)", ylabel = "Плотность")
x_dens = range(0, maximum(intervals), length=100)
theor_dens = pdf.(Exponential(1/□), x_dens)

plot!(p3, x_dens, theor_dens, label = "Экспоненциальная плотность", lw=2)
title!(p3, "Распределение интервалов между атаками")
nothing
```

4. QQ-plot интервалов против экспоненциального распределения

```
p4 = qqplot(Exponential(1/□), intervals, qqline = :identity,
xlabel = "Теоретические квантили", ylabel = "Эмпирические квантили",
title = "QQ-plot интервалов")
nothing
```

Объединяем графики

```
plot(p1, p2, p3, p4, layout = (2,2), size = (1000, 800))
nothing
```

Сохраняем графики в папку plots

```
savefig(plotsdir("attack_sim_plots.png"))
println("Графики сохранены в ", plotsdir("attack_sim_plots.png"))
```

```
Графики сохранены в C:\Users\arist\Github\2026-1--study--security-
modeling\labs\lab02\project\plots\attack_sim_plots.png
```

```julia
cd(joinpath(@__DIR__, "../project"))

# Активируем окружение проекта
using Pkg
Pkg.activate(".")
Pkg.instantiate()

# Активируем DrWatson в текущей папке
using DrWatson
@quickactivate "."
using Distributions, Plots, Statistics, Random, JLD2
gr(fmt=:png)
ENV["GKSwstype"] = "100"
```

```
  Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
```

```
"100"
```

Параметры модели

```julia
□ = 5.0
theor_prob = 1 - cdf(Poisson(□), 10)
```

```
0.013695268598382881
```

Размеры выборки для исследования (логарифмическая шкала)

```julia
sample_sizes = [10, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]
```

```
9-element Vector{Int64}:
     10
     50
    100
    500
   1000
   5000
  10000
  50000
 100000
```

Для воспроизводимости

```
Random.seed!(123)
```

```
TaskLocalRNG()
```

Вычисляем эмпирические оценки для каждого объёма выборки

```
estimates = Float64[]
println("Вычисление оценок вероятности...")
for n in sample_sizes
    hourly_sample = rand(Poisson(λ), n)
    emp_prob = count(hourly_sample .> 10) / n
    push!(estimates, emp_prob)
    println("n = $n: оценка = $emp_prob")
end
```

```
Вычисление оценок вероятности...
n = 10: оценка = 0.0
n = 50: оценка = 0.02
```

```
n = 100: оценка = 0.01
n = 500: оценка = 0.018
n = 1000: оценка = 0.014
n = 5000: оценка = 0.016
n = 10000: оценка = 0.0131
n = 50000: оценка = 0.0136
n = 100000: оценка = 0.0139
```

Построение графика сходимости

```
p = plot(sample_sizes, estimates,
         xscale = :log10,
         marker = :circle,
         label = "Эмпирическая оценка",
         xlabel = "Объём выборки (часы)",
         ylabel = "Оценка вероятности P(>10)",
         legend = :bottomright)
hline!(p, [theor_prob],
       label = "Теоретическое значение",
       ls = :dash,
       lw = 2,
       color = :red)
title!(p, "Сходимость оценки вероятности P(>10) при □=$□")
nothing
```

Сохраняем график в папку plots

```
plot_path = plotsdir("convergence.png")
savefig(p, plot_path)
println("График сохранён в $plot_path")
```

```
График сохранён в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\plots\convergence.png
```

Сохраняем данные сходимости в JLD2 (опционально)

```
data_path = datadir("convergence", "convergence_data_□=$(□).jld2")
mkpath(datadir("convergence"))
@save data_path sample_sizes estimates □ theor_prob
println("Данные сходимости сохранены в $data_path")
```

```
Данные сходимости сохранены в C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project\data\convergence\convergence_data_□=5.0.jld2
```

```
cd(joinpath(@__DIR__, "../project"))

# Активируем окружение проекта
using Pkg
Pkg.activate(".")
Pkg.instantiate()

# Активируем DrWatson в текущей папке
using DrWatson
@quickactivate "."
mkpath(datadir("parameter_sweep"))

using Distributions, Statistics, Plots, StatsPlots, JLD2, Random, CSV, DataFrames
gr(fmt=:png)
ENV["GKSwstype"] = "100"
```

```
Activating project at `C:\Users\arist\Github\2026-1--study--security-modeling\labs\lab02\project`
```

```
"100"
```

Включаем функцию симуляции

```
include(srcdir("simulation.jl"))
```

```
simulate_attacks (generic function with 2 methods)
```

Базовые параметры (общие для всех запусков)

```
base_params = Dict(
    :T => 24.0,
    :num_hours_for_est => 10000
)
```

```
Dict{Symbol, Real} with 2 entries:
  :T                 => 24.0
  :num_hours_for_est => 10000
```

Диапазон значений λ для исследования

```
□_values = [2.0, 5.0, 8.0, 12.0, 15.0]
```

```
5-element Vector{Float64}:
  2.0
  5.0
  8.0
 12.0
 15.0
```

Для воспроизводимости

```
Random.seed!(42)
```

```
TaskLocalRNG()
```

Создаём папку для графиков параметрического исследования, если её нет

```
parametric_plots_dir = plotsdir("parameter_sweep")
mkpath(parametric_plots_dir)
```

```
"C:\\Users\\arist\\Github\\2026-1--study--security-modeling\\labs\\lab02\\project\\pl
```

Структура для хранения сводных результатов

```
summary = Dict{Float64, Dict}()
```

```
Dict{Float64, Dict}()
```

Проводим параметрическое исследование для каждого лямбда

```
println("Запуск параметрического исследования...")
for □ in □_values
    params = merge(base_params, Dict(:□ => □))

    filename = datadir("attack_sim", savename(params, "jld2"))

    if isfile(filename)
        println("Загрузка существующих данных для □ = $□ из $filename")
        @load filename data
    else
        println("Выполнение симуляции для □ = $□")
        res = simulate_attacks(□, params[:T])
```

```
        hourly_sample = rand(Poisson(□), params[:num_hours_for_est])
        emp_prob = count(hourly_sample .> 10) / params[:num_hours_for_est]
        theor_prob = 1 - cdf(Poisson(□), 10)
        data = Dict(
            :hourly_counts => res.hourly_counts,
            :intervals => res.intervals,
            :attack_times => res.attack_times,
            :emp_prob => emp_prob,
            :theor_prob => theor_prob
        )

        @save filename data params
        println("Результаты сохранены в $filename")
    end


    hourly_counts = data[:hourly_counts]
    intervals = data[:intervals]
    attack_times = data[:attack_times]

    p1 = histogram(hourly_counts, bins = 0:maximum(hourly_counts), normalize = :proba
    label = "Эмпирическая частота", xlabel = "Число атак за час", ylabel = "Вероятнос
    x_vals = 0:maximum(hourly_counts)
    theor_probs = pdf.(Poisson(□), x_vals)
    plot!(p1, x_vals, theor_probs, line = :stem, marker = :circle, label = "Теоретиче
    title!(p1, "Распределение числа атак за час (□=$□)")

    p2 = plot(attack_times, 1:length(attack_times), label = "Реализация", xlabel = "В
```

```
plot!(p2, 0:0.1:params[:T], □*(0:0.1:params[:T]), label = "Среднее □·t", ls = :da
title!(p2, "Накопленное число атак в течение $(params[:T]) ч (□=$□)")


p3 = histogram(intervals, bins = 30, normalize = :pdf, label = "Эмпирическая плот
xlabel = "Интервал (ч)", ylabel = "Плотность")
x_dens = range(0, maximum(intervals), length=100)
theor_dens = pdf.(Exponential(1/□), x_dens)
plot!(p3, x_dens, theor_dens, label = "Экспоненциальная плотность", lw=2)
title!(p3, "Распределение интервалов между атаками (□=$□)")

p4 = qqplot(Exponential(1/□), intervals, qqline = :identity,
xlabel = "Теоретические квантили", ylabel = "Эмпирические квантили",
title = "QQ-plot интервалов (□=$□)")

combined = plot(p1, p2, p3, p4, layout = (2,2), size = (1000, 800))

plot_filename = joinpath(parametric_plots_dir, "attack_sim_□=$(□).png")
savefig(combined, plot_filename)

println("Детальные графики для □=$□ сохранены в $plot_filename")

summary[□] = Dict(
    :emp_prob => data[:emp_prob],
    :theor_prob => data[:theor_prob],
    :filename => filename
)
```

```
    println("□ = $□: теоретическая вероятность = $(data[:theor_prob]), эмпирическая =
end
nothing
```

```
Запуск параметрического исследования...
Загрузка существующих данных для □ = 2.0 из C:\Users\arist\Github\2026-
1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_e
Детальные графики для □=2.0 сохранены в C:\Users\arist\Github\2026-1--
study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_□=2.0.pn
□ = 2.0: теоретическая вероятность = 8.308224368480666e-6, эмпирическая = 0.0
Загрузка существующих данных для □ = 5.0 из C:\Users\arist\Github\2026-
1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_e
Детальные графики для □=5.0 сохранены в C:\Users\arist\Github\2026-1--
study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_□=5.0.pn
□ = 5.0: теоретическая вероятность = 0.013695268598382881, эмпирическая = 0.0133
Загрузка существующих данных для □ = 8.0 из C:\Users\arist\Github\2026-
1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_e
Детальные графики для □=8.0 сохранены в C:\Users\arist\Github\2026-1--
study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_□=8.0.pn
□ = 8.0: теоретическая вероятность = 0.18411420744145301, эмпирическая = 0.1822
Загрузка существующих данных для □ = 12.0 из C:\Users\arist\Github\2026-
1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_e
Детальные графики для □=12.0 сохранены в C:\Users\arist\Github\2026-1--
study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_□=12.0.p
□ = 12.0: теоретическая вероятность = 0.6527705824458284, эмпирическая = 0.6505
Загрузка существующих данных для □ = 15.0 из C:\Users\arist\Github\2026-
1--study--security-modeling\labs\lab02\project\data\attack_sim\T=24.0_num_hours_for_e
Детальные графики для □=15.0 сохранены в C:\Users\arist\Github\2026-1--
study--security-modeling\labs\lab02\project\plots\parameter_sweep\attack_sim_□=15.0.p
```

```
□ = 15.0: теоретическая вероятность = 0.8815355884709849, эмпирическая = 0.8809
```

Сохраняем сводные данные отдельно

```
summary_filename = datadir("parameter_sweep", "summary_□_values.jld2")
@save summary_filename □_values summary
println("Сводные данные сохранены в $summary_filename")
```

```
Сводные данные сохранены в C:\Users\arist\Github\2026-1--study--
security-modeling\labs\lab02\project\data\parameter_sweep\summary_□_values.jld2
```

Построение графика зависимости вероятностей от λ

```
□s = [□ for □ in □_values]
theor_probs = [summary[□][:theor_prob] for □ in □s]
emp_probs = [summary[□][:emp_prob] for □ in □s]
p = plot(□s, [theor_probs emp_probs],
        label = ["Теоретическая P(>10)" "Эмпирическая P(>10)"], marker = :circle,
        xlabel = "Интенсивность □ (атак/час)",
        ylabel = "Вероятность P(>10)",
        title = "Зависимость вероятности от интенсивности атак")
global_plot_path = plotsdir("parameter_sweep.png")
savefig(p, global_plot_path)
println("Общий график зависимости сохранён в $global_plot_path")
```

```
Общий график зависимости сохранён в C:\Users\arist\Github\2026-1--
study--security-modeling\labs\lab02\project\plots\parameter_sweep.png
```

Опционально: таблица в формате CSV

```
df = DataFrame(□ = □s, theoretical = theor_probs, empirical = emp_probs)
CSV.write(datadir("parameter_sweep", "summary.csv"), df)
println("Таблица сохранена в ", datadir("parameter_sweep", "summary.csv"))
```

```
Таблица сохранена в C:\Users\arist\Github\2026-1--study--security-
modeling\labs\lab02\project\data\parameter_sweep\summary.csv
```

7 Выводы

1. Сформирована структура рабочего пространства на основе DrWatson, обеспечивающая разделение исходных кодов, данных и документации.
2. Реализована симуляция пуассоновского потока атак двумя способами: почасовые счётчики из распределения Пуассона и точные моменты через экспоненциальные интервалы.
3. Выполнен статистический анализ результатов: гистограммы, накопленный график, QQ-plot — все подтверждают соответствие теоретическим распределениям.
4. Проведено параметрическое исследование: показана зависимость P(>10) от интенсивности λ — при росте λ от 2 до 15 вероятность возрастает от ~0 до ~0.88.
5. Исследована сходимость оценки вероятности редкого события: для надёжной оценки P(>10) при λ=5 необходим объём выборки порядка 10 000–100 000 часов.
6. Выполнена интеграция вычислительных экспериментов с их описанием за счёт преобразования кода в литературный стиль.
7. Автоматизирована генерация артефактов (чистый код, Notebook, отчёт Quarto), что повышает воспроизводимость исследования.

Список литературы

1. Описание лабораторных работ